

Quantum Feature Spaces and Kernels

QML

Contents

1	Overview	3
2	Classical Feature Maps and Kernel Trick (Deep Dive)	3
2.1	Feature Maps	3
2.2	The Kernel Function	3
2.3	Mercer's Theorem and Positive Semidefiniteness	4
2.4	The Kernel Trick in Algorithms	4
3	Quantum Feature Maps	4
3.1	The Basic Idea	4
3.2	Density Matrix Feature Map	5
3.3	Common Encoding Strategies	5
3.4	Encoding Strategies Compared	7
4	The Quantum Kernel	7
4.1	Definition	7
4.2	Why the Fidelity?	7
4.3	Computing the Quantum Kernel: The Overlap Circuit	8
4.4	Explicit Example: 2-Qubit ZZFeatureMap Kernel	8
4.5	Shot Budget for Kernel Estimation	8
5	The Full QML Kernel Workflow	9
5.1	Pipeline Diagram	9
5.2	Step by Step	9
5.3	Prediction Cost	9
6	When Quantum Kernels Help vs When They Dont	9
6.1	The Two Conditions for Quantum Advantage	9
6.2	When Quantum Kernels are Classically Simulable	10
6.3	When Quantum Kernels Might Have an Advantage	10
7	Havlíček et al. 2019: The Foundation	10
7.1	The Paper	10

7.2	Key Contributions	10
7.3	The Constructed Advantage	11
7.4	Experimental Results	11
8	Liu-Arunachalam-Temme 2021: Rigorous Advantage	11
8.1	The Paper	11
8.2	Key Result	11
8.3	Why This Matters	11
8.4	The Geometric Test	12
9	Kernel Concentration Problem	12
9.1	The Problem	12
9.2	Why It Happens	12
9.3	Consequences	13
9.4	Solutions	13
10	Connection to Barren Plateaus	13
10.1	The Unified Picture	13
10.2	The Expressibility-Utility Tradeoff	14
11	Summary	14

1 Overview

last lecture we saw that many VQCs are secretly linear classifiers in some quantum feature space. this lecture makes that precise: we develop the full theory of quantum feature maps, quantum kernels, and the connection to classical kernel methods. this is one of the cleanest and most important results in QML — it gives us a principled way to understand what quantum models actually compute, and when they might have an advantage.

the punchline: quantum computers naturally produce feature maps into exponentially large hilbert spaces, and the kernel trick lets us use these features without explicitly computing them. but (and this is the important part) having a big feature space doesn't automatically help — the features need to be relevant to the task.

we'll cover the theory from the ground up, build the key circuits, and discuss the papers that shaped this field: havlíček et al. 2019 and liu-arunachalam-temme 2021.

2 Classical Feature Maps and Kernel Trick (Deep Dive)

we covered kernels briefly in lecture 08, but now we need the full picture.

2.1 Feature Maps

Definition 2.1 (Feature Map). a feature map is a function $\phi : \mathcal{X} \rightarrow \mathcal{F}$ that maps data from input space $\mathcal{X} \subseteq \mathbb{R}^d$ to a feature space $\mathcal{F}$ (a hilbert space, possibly infinite-dimensional).

Example 2.1. polynomial feature map for $x \in \mathbb{R}^2$:

$$\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^2x_2, x_1x_2^2, x_1^2x_2^2) \in \mathbb{R}^9$$

maps 2D data into 9D. a linear classifier in this 9D space can represent quadratic decision boundaries in the original 2D space.

Example 2.2. gaussian (RBF) feature map: this one is infinite-dimensional!

$$\phi(x) = e^{-\|x\|^2/2\sigma^2} \cdot \left(1, \frac{x}{\sigma}, \frac{x^{\otimes 2}}{\sigma^2\sqrt{2!}}, \frac{x^{\otimes 3}}{\sigma^3\sqrt{3!}}, \dots\right)$$

u can't compute $\phi(x)$ explicitly (infinite vector). but u CAN compute the inner product:

$$\langle \phi(x), \phi(x') \rangle = e^{-\|x-x'\|^2/2\sigma^2}$$

that's the kernel trick.

2.2 The Kernel Function

Definition 2.2 (Kernel Function). a kernel function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ computes inner products in feature space:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{F}}$$

why this is powerful: if the feature space $\mathcal{F}$ is exponentially large (or infinite), computing $\phi(x)$ directly is intractable. but if we can evaluate $k(x, x')$ efficiently, we can do linear classification in that space without ever touching the full feature vector.

common classical kernels:

- linear: $k(x, x') = x^\top x'$ (no feature map, just original features)
- polynomial: $k(x, x') = (x^\top x' + c)^d$ (degree- d polynomial features)
- RBF/gaussian: $k(x, x') = e^{-\|x-x'\|^2/2\sigma^2}$ (infinite-dimensional feature space)
- laplacian: $k(x, x') = e^{-\|x-x'\|_1/\sigma}$

2.3 Mercer's Theorem and Positive Semidefiniteness

not every function $k(x, x')$ is a valid kernel.

Theorem 2.1 (Mercer's Theorem (Informal)). a symmetric function $k: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a valid kernel if and only if the kernel matrix (gram matrix) $K_{ij} = k(x_i, x_j)$ is positive semidefinite (PSD) for any finite set of points $\{x_1, \dots, x_N\}$.

PSD means: for all $v \in \mathbb{R}^N$, $v^\top K v \geq 0$. equivalently, all eigenvalues of K are non-negative.

the gram matrix for N data points:

$$K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{pmatrix}$$

Key Point. any inner product in a hilbert space automatically gives a PSD kernel. this means: quantum feature maps $\phi(x) = \rho(x)$ with hilbert-schmidt inner product $\text{Tr}[\rho(x)\rho(x')]$ always define valid kernels. u dont need to check mercer's condition — its guaranteed by quantum mechanics.

2.4 The Kernel Trick in Algorithms

algorithms that depend on data only through inner products $\langle \phi(x_i), \phi(x_j) \rangle$ can be “kernelized”:

- SVM: dual formulation uses $k(x_i, x_j)$ — the classic example
- kernel ridge regression: $f(x) = k(x, \cdot)^\top (K + \lambda I)^{-1} y$
- kernel PCA: eigendecompose the centered gram matrix
- gaussian processes: the kernel IS the prior covariance

3 Quantum Feature Maps

3.1 The Basic Idea

a quantum feature map encodes classical data x into a quantum state:

Definition 3.1 (Quantum Feature Map). a quantum feature map is a map $\phi_Q: \mathcal{X} \rightarrow \mathcal{H}_{2^n}$ defined by:

$$x \mapsto |\phi(x)\rangle = U_\phi(x) |0\rangle^{\otimes n}$$

where $U_\phi(x)$ is a data-dependent unitary circuit and $\mathcal{H}_{2^n}$ is the 2^n -dimensional hilbert space of n qubits.

with n qubits, the state space is 2^n -dimensional. thats exponential in the number of qubits:

qubits n	hilbert space dim 2^n
4	16
10	1,024
20	1,048,576
50	$\sim 10^{15}$
100	$\sim 10^{30}$

for comparison, the number of atoms in the visible universe is $\sim 10^{80}$. a 300-qubit feature map has more dimensions than atoms.

3.2 Density Matrix Feature Map

theres a subtlety: pure states $|\phi(x)\rangle$ have global phase ambiguity ($e^{i\alpha}|\phi(x)\rangle$ represents the same physical state). the more natural object is the density matrix:

Definition 3.2 (Density Matrix Feature Map).

$$x \mapsto \rho(x) = |\phi(x)\rangle \langle \phi(x)| = U_\phi(x) |0\rangle \langle 0|^{\otimes n} U_\phi^\dagger(x)$$

this lives in the space of $2^n \times 2^n$ hermitian matrices, which has real dimension 4^n (as opposed to 2^n for pure states).

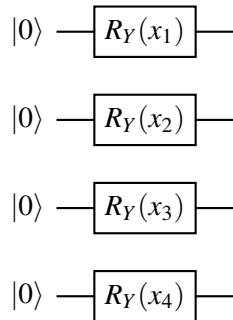
the feature space for density matrices is even bigger: 4^n real dimensions. for 10 qubits thats $\sim 10^6$ features. the inner product in this space is the hilbert-schmidt inner product:

$$\langle \rho(x), \rho(x') \rangle_{HS} = \text{Tr}[\rho(x)\rho(x')]$$

3.3 Common Encoding Strategies

lets go through the main quantum feature maps used in practice.

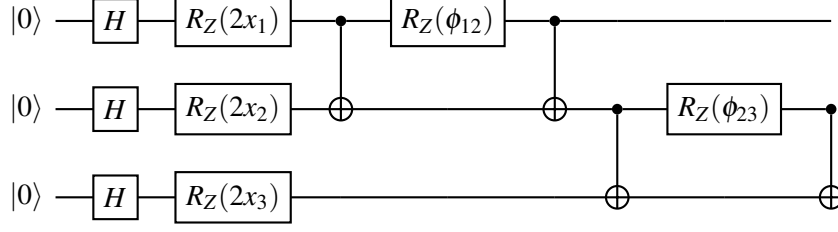
1. angle encoding:



each qubit encodes one feature as a rotation angle. feature space is a tensor product of 2D subspaces — 2^n dimensional but highly structured. no entanglement means the quantum features r just products of single-qubit features. this is classically easy to simulate.

2. ZZFeatureMap (havlíček encoding):

for 3 qubits, 1 rep:



where $\phi_{ij} = 2(\pi - x_i)(\pi - x_j)$.

this creates entanglement through ZZ interactions. the feature space includes cross-terms between features — u cant decompose it into single-qubit contributions. this is the encoding that started the quantum kernel revolution.

3. PauliFeatureMap (generalized):

$$U_\phi(x) = \prod_{\ell=1}^r \left[\exp \left(i \sum_S \phi_S(x) \bigotimes_{j \in S} P_j \right) H^{\otimes n} \right]$$

the S ranges over subsets of qubits, $P_j \in \{Z, Y, X\}$, and $\phi_S(x) = \prod_{j \in S} x_j$.

options:

- $|S| = 1, P = Z$: ZFeatureMap (diagonal, no entanglement)
- $|S| \leq 2, P = Z$: ZZFeatureMap (pairwise entanglement)
- $|S| \leq 3$, mixed P : higher-order interactions, deeper circuits

4. amplitude encoding:

$$|\phi(x)\rangle = \frac{1}{\|x\|} \sum_{i=0}^{2^n-1} x_i |i\rangle$$

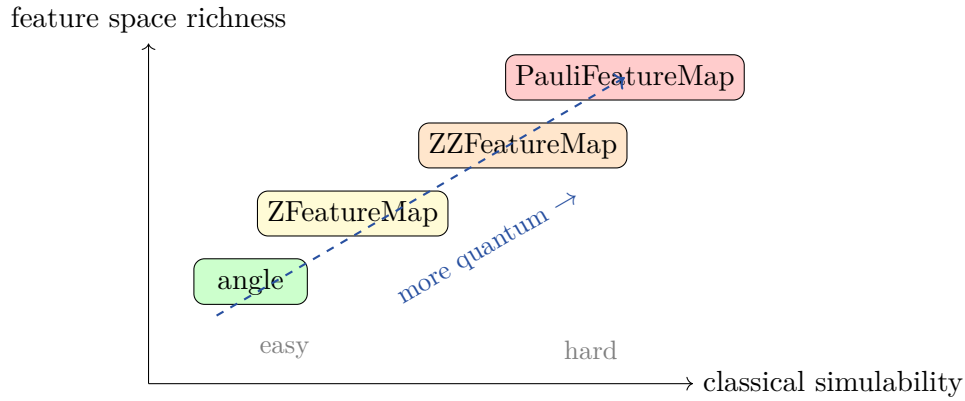
encodes 2^n classical features in the amplitudes of n qubits. exponentially efficient in space, but state preparation requires $O(2^n)$ gates in general.

5. IQP-style encoding:

$$U_\phi(x) = \text{diag}(\exp(if(x))) \cdot H^{\otimes n}$$

where $f(x)$ is a polynomial in the features. IQP (instantaneous quantum polynomial) circuits r diagonal in the Z basis. they r conjectured to be classically hard to simulate, which is important for quantum advantage.

3.4 Encoding Strategies Compared



4 The Quantum Kernel

4.1 Definition

Definition 4.1 (Quantum Kernel (Fidelity Kernel)). the quantum kernel between two data points x, x' is:

$$\kappa(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2 = \text{Tr}[\rho(x)\rho(x')]$$

this is the transition probability (fidelity) between the two encoded quantum states.

properties:

- $\kappa(x, x) = 1$ for all x (a state always has fidelity 1 with itself)
- $0 \leq \kappa(x, x') \leq 1$ (fidelity is bounded)
- $\kappa(x, x') = \kappa(x', x)$ (symmetric)
- the gram matrix $K_{ij} = \kappa(x_i, x_j)$ is PSD (its a valid kernel by mercer's theorem)

Key Point. every quantum encoding circuit automatically defines a valid kernel. u dont need to check mercer's condition — its guaranteed by the hilbert space inner product structure. this is a beautiful fact: the physics gives u the math for free.

4.2 Why the Fidelity?

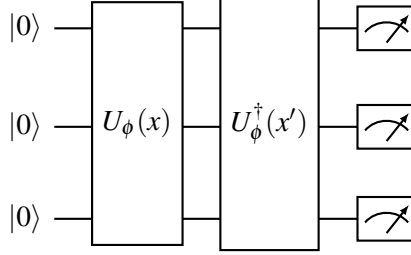
the quantum kernel $\kappa(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$ measures how “similar” the two encoded states are:

- $\kappa = 1$: states are identical (same point in feature space)
- $\kappa = 0$: states are orthogonal (maximally different)
- κ close to 1: data points are mapped nearby in hilbert space
- κ close to 0: data points are mapped far apart

for classification: we want same-class points to have high κ and different-class points to have low κ . if the feature map achieves this, a kernel SVM will find a good classifier.

4.3 Computing the Quantum Kernel: The Overlap Circuit

to compute $\kappa(x, x') = |\langle 0|^{\otimes n} U_\phi^\dagger(x') U_\phi(x) |0\rangle^{\otimes n}|^2$, we use this circuit:



apply $U_\phi(x)$ to prepare $|\phi(x)\rangle$, then apply the inverse $U_\phi^\dagger(x')$. if $x = x'$, this gives $U_\phi^\dagger(x) U_\phi(x) = I$, so we get $|0\rangle^{\otimes n}$ with certainty.

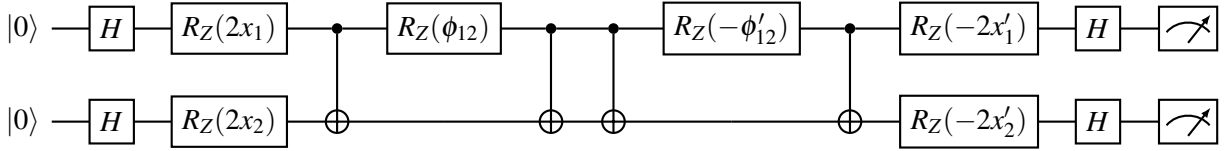
measure all qubits in the computational basis. the probability of getting the all-zeros outcome is:

$$P(00\dots 0) = |\langle 0|^{\otimes n} U_\phi^\dagger(x') U_\phi(x) |0\rangle^{\otimes n}|^2 = \kappa(x, x')$$

Key Point. the kernel is estimated by running the overlap circuit many times and counting how often u get the all-zeros outcome. the fraction of all-zeros measurements gives $\hat{\kappa}(x, x')$.

4.4 Explicit Example: 2-Qubit ZZFeatureMap Kernel

the overlap circuit for ZZFeatureMap on 2 qubits:



where $\phi_{12} = 2(\pi - x_1)(\pi - x_2)$ and $\phi'_{12} = 2(\pi - x'_1)(\pi - x'_2)$. this is the circuit from havlíček et al. 2019.

4.5 Shot Budget for Kernel Estimation

each kernel entry $\hat{\kappa}(x, x')$ is estimated from S shots:

$$\hat{\kappa}(x, x') = \frac{\text{number of all-zero outcomes}}{S}$$

this is a binomial random variable with mean $\kappa(x, x')$ and variance:

$$\text{Var}[\hat{\kappa}] = \frac{\kappa(1 - \kappa)}{S}$$

to achieve additive error ε with high probability: need $S = O(1/\varepsilon^2)$ shots.

for the full kernel matrix of N training points: need to estimate $\binom{N}{2}$ entries (symmetric matrix, diagonal is 1). total cost:

$$\text{total shots} = O\left(\frac{N^2}{\varepsilon^2}\right)$$

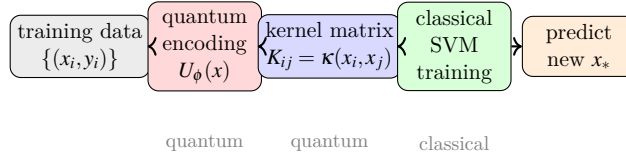
Example 4.1. $N = 100$ training points, $\varepsilon = 0.01$ accuracy per entry:

$$\text{total circuits} = \frac{100 \times 99}{2} \times \frac{1}{0.01^2} = 4,950 \times 10,000 = 49,500,000$$

thats ~ 50 million circuit executions. at 10,000 circuits/second on current hardware ≈ 83 minutes. this is the main scalability bottleneck of quantum kernel methods.

5 The Full QML Kernel Workflow

5.1 Pipeline Diagram



5.2 Step by Step

1. choose feature map: pick $U_\phi(x)$ (ZZFeatureMap, PauliFeatureMap, etc.)
2. compute kernel matrix: for all pairs (x_i, x_j) in training set, run the overlap circuit and estimate $\kappa(x_i, x_j)$. result: $N \times N$ matrix K .
3. train SVM: use K as the kernel matrix in a classical SVM. this is a convex optimization problem — no local minima, guaranteed global optimum. u get support vectors and dual coefficients α_i .
4. predict: for new point x_* , compute $\kappa(x_*, x_i)$ for all support vectors x_i . the prediction is:

$$f(x_*) = \text{sign} \left(\sum_{i \in \text{SV}} \alpha_i y_i \kappa(x_i, x_*) + b \right)$$

5.3 Prediction Cost

for a new test point x_* :

- need to compute $\kappa(x_*, x_i)$ for each support vector x_i
- number of support vectors: typically $\sim 10\text{--}30\%$ of training set
- each kernel evaluation: $O(1/\varepsilon^2)$ shots
- prediction cost per test point: $O(N_{\text{SV}}/\varepsilon^2)$

6 When Quantum Kernels Help vs When They Dont

6.1 The Two Conditions for Quantum Advantage

Key Point. a quantum kernel provides a genuine advantage only when BOTH conditions hold:

1. computational hardness: the kernel $\kappa(x, x')$ cannot be efficiently computed classically
2. task relevance: the kernel is useful for the classification task (high kernel-target alignment)

either condition alone is insufficient. a classically hard kernel that doesn't separate the data is useless. a great kernel that can be computed classically needs no quantum computer.

6.2 When Quantum Kernels are Classically Simulable

the quantum kernel is classically easy when:

- no entanglement: tensor product encodings (angle encoding, ZZFeatureMap). the kernel factorizes: $\kappa(x, x') = \prod_i \kappa_i(x_i, x'_i)$. each factor is a simple function.
- clifford circuits: Gottesman-Knill theorem — clifford circuits can be simulated in polynomial time. if the feature map is clifford, the kernel is classical.
- low entanglement: if the entanglement entropy stays bounded, matrix product state methods can compute the kernel efficiently.
- shallow circuits: depth- $O(\log n)$ circuits with local gates can be contracted classically using tensor network methods.

6.3 When Quantum Kernels Might Have an Advantage

- deep entangling circuits: ZZFeatureMap with multiple reps and all-to-all connectivity. these create long-range entanglement that's hard to simulate classically.
- IQP-type encodings: conjectured classically hard under standard complexity assumptions (polynomial hierarchy doesn't collapse).
- specific problem structures: problems with hidden algebraic structure (like discrete log) where quantum algorithms have proven advantages.

7 Havlíček et al. 2019: The Foundation

7.1 The Paper

“Supervised learning with quantum-enhanced feature spaces” (Nature, 2019). this is THE paper that established quantum kernel methods.

7.2 Key Contributions

1. defined the quantum kernel $\kappa(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$
2. proposed the ZZFeatureMap encoding
3. showed how to estimate the kernel on a quantum computer
4. demonstrated it on a 2-qubit IBM quantum device
5. proved that for certain data distributions, the quantum kernel provides a classification advantage

7.3 The Constructed Advantage

havlíček et al. constructed a specific binary classification problem where:

- the data is generated by a process related to the discrete logarithm
- the quantum kernel with ZZFeatureMap perfectly separates the classes
- any classical kernel with polynomial-dimensional feature space fails

Intuition. the idea is simple: design the dataset so that the “correct” decision boundary lives in a quantum feature space that classical models cant access. then the quantum kernel wins by construction.

the catch: this is an artificial dataset designed for quantum advantage. the open question is whether natural datasets have this structure.

7.4 Experimental Results

on 2 qubits (ibm device):

- quantum kernel SVM: $\sim 100\%$ accuracy on the constructed dataset
- classical SVM (various kernels): $\sim 50\%$ (random guessing)
- this was one of the first experimental demonstrations of a quantum ML advantage

8 Liu-Arunachalam-Temme 2021: Rigorous Advantage

8.1 The Paper

“A rigorous and robust quantum speed-up in supervised machine learning” (Nature Physics, 2021). this paper put the advantage on rigorous footing.

8.2 Key Result

Theorem 8.1 (Liu et al. 2021 — Informal). there exist classification problems where:

1. a quantum kernel method achieves high accuracy with polynomial samples and time
2. any classical ML method (not just kernel methods — ANY classical algorithm) requires superpolynomial resources

the advantage holds under standard cryptographic assumptions (hardness of learning with errors).

8.3 Why This Matters

the havlíček result showed advantage over classical kernel methods. liu et al. showed advantage over all classical methods. thats a much stronger statement.

but: the problems are still artificial (based on cryptographic constructions). no one has shown this kind of advantage for a natural, practically relevant classification task.

8.4 The Geometric Test

liu et al. also introduced a practical diagnostic: the geometric difference:

$$g(K_C, K_Q) = \sqrt{\left\| \sqrt{K_Q^{-1}} K_C \sqrt{K_Q^{-1}} \right\|_\infty}$$

where K_C is the best classical kernel matrix and K_Q is the quantum kernel matrix.

- if $g \gg 1$: the quantum kernel captures structure that the classical kernel misses. potential for quantum advantage.
- if $g \approx 1$: the two kernels are similar. no quantum advantage expected.

this gives u a practical test: before committing to quantum kernel methods, compute g and check if its worth it.

9 Kernel Concentration Problem

9.1 The Problem

Definition 9.1 (Kernel Concentration). a quantum kernel exhibits concentration when:

$$\kappa(x, x') \approx \delta_{x, x'} \cdot 1 + (1 - \delta_{x, x'}) \cdot \frac{1}{2^n}$$

for most pairs (x, x') . that is, the kernel value for distinct points approaches $1/2^n \approx 0$.

Intuition. in a 2^n -dimensional space, random vectors are nearly orthogonal. if the feature map distributes points uniformly (like a random circuit would), all pairwise inner products are exponentially small. the kernel matrix approaches the identity matrix, which carries no information about data structure.

9.2 Why It Happens

for an n -qubit system, the hilbert space has 2^n dimensions. by concentration of measure:

$$\mathbb{E}_{x \neq x'}[\kappa(x, x')] = O(2^{-n})$$

this happens when:

- the feature map is highly expressive (forms a 2-design or close to it)
- many qubits with deep entangling circuits
- the data fills the feature space somewhat uniformly

9.3 Consequences

if $\kappa(x, x') \approx 0$ for all $x \neq x'$, the gram matrix $K \approx I$. then:

- the SVM has no structure to exploit — all points look equally different
- the classifier essentially memorizes the training set (each point is its own support vector)
- generalization is terrible bcs the kernel provides no useful inductive bias

this is the kernel analog of barren plateaus: both r manifestations of “too much quantum” making the model useless.

9.4 Solutions

1. projected quantum kernels (huang et al. 2021): project onto reduced density matrices instead of using the full state:

$$\kappa_{\text{proj}}(x, x') = \exp \left(-\gamma \sum_i \|\rho_i(x) - \rho_i(x')\|_F^2 \right)$$

where $\rho_i(x) = \text{Tr}_i[\rho(x)]$ is the reduced density matrix of qubit i .

2. fewer reps: use ZZFeatureMap with 1 rep instead of 2+. less expressibility = less concentration.
3. bandwidth tuning: scale the data: $x \rightarrow \gamma x$. smaller γ means smaller rotations, less expressive feature map, less concentration.
4. fewer qubits: use dimensionality reduction (PCA) to reduce features before encoding. fewer qubits = less concentration.

10 Connection to Barren Plateaus

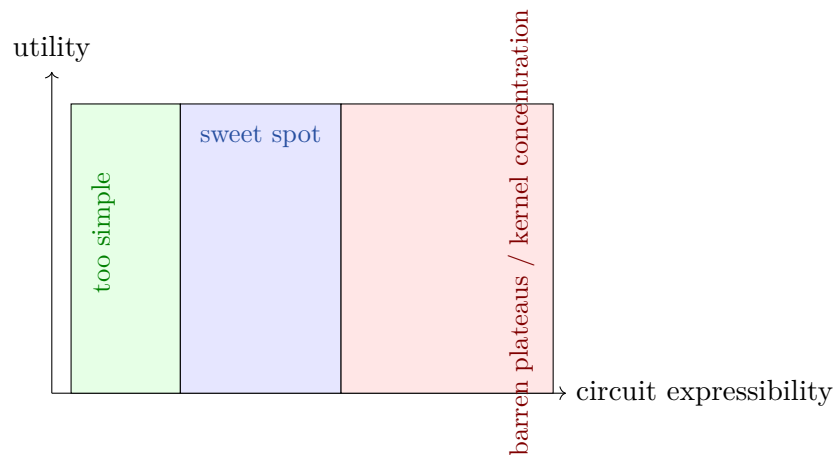
10.1 The Unified Picture

thanasilp, tangpanitanon, and coles (2024) showed that kernel concentration and barren plateaus r closely connected:

Theorem 10.1 (Thanasilp et al. 2024 — Informal). if a parameterized quantum circuit exhibits a barren plateau (exponentially vanishing gradients), then the associated quantum kernel also concentrates (exponentially small off-diagonal entries).

Key Point. barren plateaus and kernel concentration r two sides of the same coin. both come from the circuit being “too expressive” (too close to a 2-design). avoiding one helps avoid the other. the common fix: use structured, problem-aware circuits instead of random deep circuits.

10.2 The Expressibility-Utility Tradeoff



the sweet spot: expressive enough to capture task-relevant features, but not so expressive that the landscape becomes flat.

11 Summary

key takeaways:

1. quantum feature maps encode classical data into 2^n -dimensional (state) or 4^n -dimensional (density matrix) hilbert spaces
2. the quantum kernel $\kappa(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$ is always a valid kernel (PSD guaranteed by physics)
3. computing the kernel requires the overlap circuit + measurement, with cost $O(N^2/\epsilon^2)$ for the full gram matrix
4. the kernel workflow: encode $\rightarrow$ compute kernel matrix $\rightarrow$ train classical SVM $\rightarrow$ predict
5. quantum advantage requires BOTH classical hardness AND task relevance
6. havlíček et al. 2019 established the framework; liu et al. 2021 proved rigorous advantage (on artificial problems)
7. kernel concentration: in large systems, kernels approach the identity matrix — no useful structure
8. kernel concentration and barren plateaus r two sides of the same coin (thanasilp et al. 2024)
9. practical fixes: projected kernels, fewer reps, bandwidth tuning, dimensionality reduction

the quantum kernel framework is theoretically beautiful and gives us the tools to rigorously analyze quantum ML models. but it also reveals the fundamental challenge: the exponentially large feature space is a double-edged sword.

next lecture: we put this theory into practice — implementing quantum kernels in qiskit, computing kernel matrices, and running quantum SVMs.